

Plan de Proyecto: Simulador Instinct

1. Comprensión General del Proyecto

Antes de escribir código, déjame resumir qué es lo que hay que construir:

Un simulador de ecosistemas donde:

- Criaturas programadas en un lenguaje propio (.ins) viven en un mapa
- Tienen un ciclo de vida basado en ticks (turnos)
- Interactúan entre sí mediante acciones (moverse, atacar, consumir, reproducirse, etc.)
- El usuario carga archivos, coloca criaturas y observa la simulación

2. Componentes Principales del Proyecto

text

INSTINCT SIMULATOR

└─ 1. PARSER/COMPILADOR del lenguaje Instinct

| └─ Lexer (tokenizador)

| └─ Parser (análisis sintáctico)

| └─ AST (árbol de sintaxis abstracta)

|

└─ 2. INTÉRPRETE

| └─ Evaluador de expresiones

| └─ Ejecutor de acciones

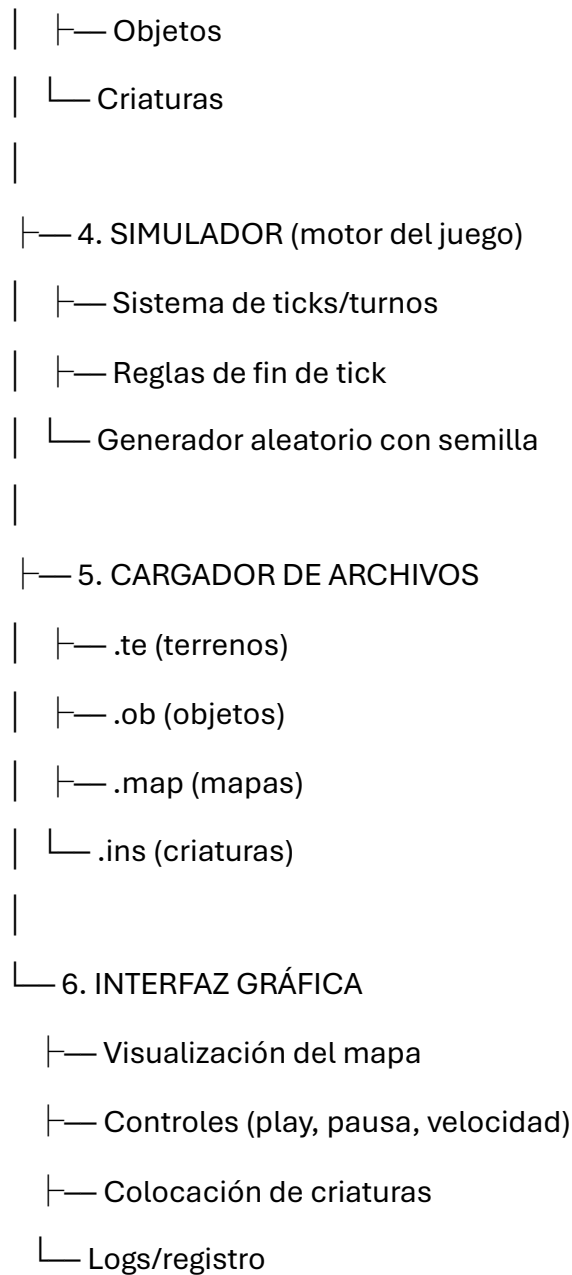
| └─ Gestor de percepciones

|

└─ 3. MODELO DEL MUNDO

| └─ Mapa (grid de casillas)

| └─ Terrenos



3. Jerarquías de Clases (Diseño OOP)

Este es el corazón del proyecto desde el punto de vista de la evaluación. El PDF enfatiza mucho esto.

3.1 Lo que vive en el mundo

text

WorldElement (abstracta)

└─ name: str

└─ position: (x, y)

|

└─ Terrain

| └─ resource_max: int

| └─ regen: int

| └─ reserve: int (actual)

| └─ char: str

|

└─ WorldObject

| └─ resource_max: int

| └─ reserve: int (actual)

| └─ char: str

|

└─ Creature

└─ species: str

└─ faction: str

└─ health: int (actual)

└─ max_health_initial: int

└─ vision: int

└─ lifespan: int

└─ age: int

└─ position: (x, y)

└─ variables: dict

└─ program: CompiledProgram

└─ instruction_pointer: int

└─ wait_remaining: int

└─ alive: bool

3.2 Acciones (Polimorfismo clave)

text

Action (abstracta)

└─ validate(args): comprueba número y tipo de argumentos

└─ execute(creature, world, args): aplica el efecto

└─ consumes_turn: bool

|

└─ MoveAction # move(dx, dy, speed)

└─ AttackAction # attack(dx, dy, damage)

└─ ConsumeAction # consume(dx, dy, damage, heal)

└─ ReproduceAction # reproduce(dx, dy, health)

└─ RoarAction # roar(value)

└─ SayAction # say(value) ← no consume turno

└─ WaitAction # wait(n)

Esto es lo que el PDF dice que NO quiere ver:

Python

❌ MAL - cadena de ifs

if tipo == "move":

 # 30 líneas

elif tipo == "attack":

 # 30 líneas

elif tipo == "consume":

30 líneas

Lo que **Sí** quiere ver:

Python

 BIEN - polimorfismo

```
action_registry = {  
    "move": MoveAction(),  
    "attack": AttackAction(),  
    "consume": ConsumeAction(),  
    # ...  
}
```

En el intérprete:

```
action = action_registry[action_name]  
action.validate(args)  
action.execute(creature, world, evaluated_args)
```

3.3 Expresiones (AST)

text

Expression (abstracta)

```
|— evaluate(context) → valor  
|  
|— LiteralInt      # 42, -7  
|— LiteralString   # "hello"  
|— VariableRef     # mis_variable, health, enemy_dist  
|— PerceptionRef   # health, x, y, enemy_dist...  
|— ConstantRef     # NONE, GROUND, OBJECT, ALLY, ENEMY  
|
```

|— BinaryOp (abstracta)

| |— AddOp # +

| |— SubOp # -

| |— MulOp # *

| |— DivOp # /

| |— ModOp # %

| |— LessThan # <

| |— LessEqual # <=

| |— GreaterThan # >

| |— GreaterEqual # >=

| |— Equal # ==

| |— NotEqual # !=

| |— AndOp # and

| |— OrOp # or

|

|— UnaryOp (abstracta)

| |— NegateOp # -x

| |— NotOp # not x

|

|— FunctionCall

|— SeeFunction # see(x, y)

|— NameFunction # name(x, y)

3.4 Instrucciones (líneas del cuerpo)

text

Instruction (abstracta)

```

└─ line_number: int
|
└─ LabelInst      # name:
└─ GotoInst       # goto name
└─ ConditionalGotoInst # if expr goto name
└─ AssignmentInst # name = expr
└─ ActionInst     # action(args...)
└─ CommentInst    # (se descarta en compilación)

```

3.5 Percepciones

text

Perception (abstracta)

```

└─ name: str
└─ get_value(creature, world) → int | str
|
└─ HealthPerception  # health
└─ AgePerception     # age
└─ LifespanPerception # lifespan
└─ VisionPerception  # vision
└─ SpeciesPerception # species
└─ FactionPerception # faction
└─ XPerception       # x
└─ YPerception       # y
└─ TerrainHerePerception # terrain_here
└─ ReserveHerePerception # reserve_here
└─ WidthPerception   # width

```

- └─ HeightPerception # height
- └─ TickPerception # tick
- └─ TotalCreaturesPerception
- └─ TotalAlliesPerception
- └─ TotalEnemiesPerception
- └─ RandomPerception # random (nuevo valor cada lectura)
- └─ AlliesNearPerception
- └─ EnemiesNearPerception
- └─ ObjectsNearPerception
- └─ EnemyDistPerception
- └─ EnemyDxPerception
- └─ EnemyDyPerception
- └─ AllyDistPerception
- └─ AllyDxPerception
- └─ AllyDyPerception
- └─ ObjectDistPerception
- └─ ObjectDxPerception
- └─ ObjectDyPerception
- └─ RoarsNearPerception
- └─ LastRoarPerception

Registradas en un diccionario, no en una cadena de ifs:

Python

```
perception_registry = {
    "health": HealthPerception(),
    "age": AgePerception(),
```

```
"x": XPerception(),  
  
# ... etc  
}
```

4. Estructura de Directorios del Proyecto

text

instinct/

```
|  
|  
| └─ main.py          # Punto de entrada  
|  
| └─ data/            # Archivos del juego  
|   | └─ terrains/  
|   |   | └─ grass.te  
|   |   |  
|   |   └─ objects/  
|   |       | └─ rock.ob  
|   |       |  
|   |       └─ maps/  
|   |           | └─ plain_with_rocks.map  
|   |           |  
|   |           └─ creatures/  
|   |               | └─ hobbit.ins  
|   |               | └─ uruk.ins  
|   |               | └─ ent.ins  
|   |               └─ dwarf.ins  
|   |  
|   └─ src/  
|       | └─ __init__.py
```

```

| |
| | └─ language/      # Todo lo del lenguaje Instinct
| |   └─ __init__.py
| |   └─ lexer.py      # Tokenizador
| |   └─ parser.py     # Parser → AST
| |   └─ ast_nodes.py  # Clases del AST (Expression, Instruction)
| |   └─ interpreter.py # Ejecuta el AST turno a turno
| |   └─ actions.py    # Jerarquía de acciones
| |   └─ functions.py   # see(), name() y futuras
| |   └─ perceptions.py # Jerarquía de percepciones
| |   └─ constants.py  # NONE, GROUND, OBJECT, ALLY, ENEMY
| |   └─ errors.py     # CompileError, RuntimeError
| |
| | └─ world/          # El modelo del mundo
| |   └─ __init__.py
| |   └─ world.py      # La clase World (grid, lógica global)
| |   └─ cell.py       # Casilla individual
| |   └─ terrain.py    # Clase Terrain
| |   └─ world_object.py # Clase WorldObject
| |   └─ creature.py   # Clase Creature (estado, no lógica del lenguaje)
| |   └─ map_loader.py # Carga .map
| |
| | └─ loaders/        # Cargadores de archivos
| |   └─ __init__.py
| |   └─ terrain_loader.py # Carga .te

```

- | | |— object_loader.py # Carga .ob
- | | |— creature_loader.py # Carga .ins (usa lexer + parser)
- | | |— loader_manager.py # Orquesta la carga en orden
- | |
- | |— simulation/ # Motor de simulación
- | | |— __init__.py
- | | |— simulator.py # Bucle de ticks, orden de turnos
- | | |— rng.py # Generador aleatorio con semilla
- | |
- | |— ui/ # Interfaz gráfica
- | | |— __init__.py
- | | |— app.py # Ventana principal
- | | |— map_view.py # Visualización del grid
- | | |— controls.py # Play, pausa, velocidades
- | | |— creature_placer.py # Fase de preparación
- | | |— log_panel.py # Panel de logs
- |
- |— tests/ # Tests
- | |— test_lexer.py
- | |— test_parser.py
- | |— test_interpreter.py
- | |— test_actions.py
- | |— test_perceptions.py
- | |— test_world.py
- | |— test_simulation.py

└─ test_loaders.py

5. Planificación por Fases

FASE 1: El Lenguaje (lo más complejo — empezar aquí)

Semana 1-2: Lexer + Parser + AST

Tarea	Descripción	Prioridad
1.1	Definir tokens (keywords, operadores, literales, identificadores)	● Alta
1.2	Implementar Lexer: texto → lista de tokens	● Alta
1.3	Definir todas las clases del AST (ast_nodes.py)	● Alta
1.4	Implementar Parser de la cabecera	● Alta
1.5	Implementar Parser del cuerpo: cada tipo de línea	● Alta
1.6	Implementar Parser de expresiones con precedencia	● Alta
1.7	Validaciones de compilación (etiquetas, cabecera completa, etc.)	● Alta
1.8	Tests: parsear hobbit.ins, uruk.ins, ent.ins, dwarf.ins	● Alta

Semana 2-3: Intérprete

Tarea	Descripción	Prioridad
2.1	Evaluador de expresiones (recorre el AST)	● Alta
2.2	Sistema de percepciones (registry, no ifs)	● Alta
2.3	Sistema de constantes	● Alta
2.4	Funciones see() y name()	● Alta
2.5	Ejecutor de instrucciones (goto, if goto, assign)	● Alta
2.6	Sistema de acciones (registry + jerarquía)	● Alta
2.7	Implementar las 7 acciones con sus efectos	● Alta

Tarea	Descripción	Prioridad
2.8	Límite de 100 líneas por turno	● Alta
2.9	Errores de runtime (división por 0, tipos, etc.)	● Alta
2.10	Tests del intérprete con criaturas de ejemplo	● Alta

FASE 2: El Mundo

Semana 3-4

Tarea	Descripción	Prioridad
3.1	Clase Terrain y cargador .te	● Alta
3.2	Clase WorldObject y cargador .ob	● Alta
3.3	Clase World (grid, consultas, modificaciones)	● Alta
3.4	Cargador de mapas .map	● Alta
3.5	Clase Creature (estado en el mundo)	● Alta
3.6	Cálculo de distancia (Chebyshev: $\max(dx , dy)$)	dx
3.7	Tests: cargar mapa, colocar criaturas, consultar casillas	● Alta

FASE 3: El Simulador

Semana 4-5

Tarea	Descripción	Prioridad
4.1	Generador aleatorio con semilla (rng.py)	● Alta
4.2	Bucle de ticks: sortear orden, dar turnos	● Alta
4.3	Actualizar percepciones al inicio de cada turno	● Alta
4.4	Ejecutar turno de criatura (llamar al intérprete)	● Alta
4.5	Fin de tick: -1 vida, +1 edad, muertes, regeneración, objetos rotos	● Alta
4.6	Wait: saltar n-1 turnos siguientes	● Alta

Tarea	Descripción	Prioridad
4.7	Roar: buffer de rugidos del tick anterior	● Alta
4.8	Test completo: simulación en consola sin GUI	● Alta

FASE 4: Interfaz Gráfica

Semana 5-6

Tarea	Descripción	Prioridad
5.1	Elegir biblioteca (pygame, tkinter, pygamelet, etc.)	● Media
5.2	Ventana principal con layout	● Alta
5.3	Renderizado del mapa (grid con colores/chars)	● Alta
5.4	Fase de preparación: elegir mapa, colocar criaturas	● Alta
5.5	Controles: play, pausa, tick único, 3+ velocidades	● Alta
5.6	Botón de reiniciar	● Alta
5.7	Panel de logs	● Alta
5.8	Integrar simulador con la GUI (game loop)	● Alta

FASE 5: Pulido y Extensiones

Semana 6-7

Tarea	Descripción	Prioridad
6.1	Probar con TODOS los archivos de ejemplo del profesor	● Alta
6.2	Manejo robusto de errores (nada explota)	● Alta
6.3	Extensiones del lenguaje (nuevas acciones, percepciones)	● Extra
6.4	Nuevos terrenos y objetos con comportamiento	● Extra
6.5	Inspector de criatura al hacer clic	● Extra
6.6	Gráficas de población	● Extra

Tarea	Descripción	Prioridad
6.7	Criaturas propias con temática personal	● Extra

6. Detalles Técnicos Importantes a No Olvidar

Tokenización y Parsing

text

Tokens que necesitas reconocer:

- Palabras clave: if, goto, and, or, not
- Palabras de cabecera: creature, faction, health, vision, lifespan
- Constantes: NONE, GROUND, OBJECT, ALLY, ENEMY
- Operadores: +, -, *, /, %, <, <=, >, >=, ==, !=
- Delimitadores: (,), ,, :
- Literales: enteros (42, -7), strings ("hello")
- Identificadores: todo lo demás (nombres de variables, acciones, percepciones)
- Comentarios: # hasta fin de línea

Precedencia de operadores (mayor a menor)

text

1. Paréntesis, llamadas a función
2. Menos unario, not
3. *, /, %
4. +, -
5. <, <=, >, >=, ==, !=
6. and
7. or

Distancia (Chebyshev)

Python

```
def distance(x1, y1, x2, y2):  
    return max(abs(x2 - x1), abs(y2 - y1))
```

Dirección hacia un objetivo

Python

```
def direction(from_x, from_y, to_x, to_y):  
    dx = 0 if to_x == from_x else (1 if to_x > from_x else -1)  
    dy = 0 if to_y == from_y else (1 if to_y > from_y else -1)  
    return dx, dy
```

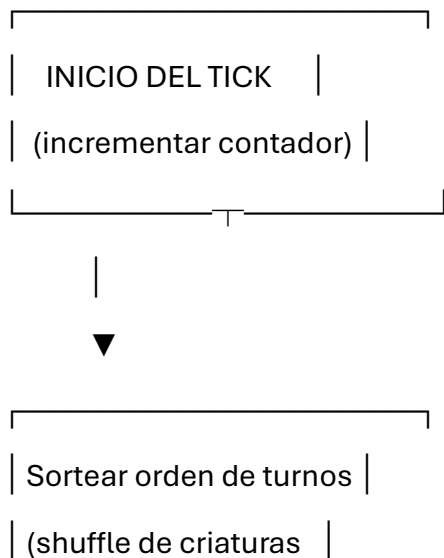
Reglas de fin de tick (en orden)

text

1. Toda criatura: health -= 1, age += 1
2. Criaturas con health <= 0 o age >= lifespan → mueren
3. Todo terreno: reserve = min(reserve + regen, resource_max)
4. Objetos con reserve <= 0 → desaparecen

7. Diagrama de Flujo de un Tick

text



| vivas usando el RNG) |

|

|



|

| ¿Quedan turnos |—No—|

| por dar? | |

|

| Sí |



|

| Tomar siguiente | |

| criatura | |

|

|



|

| ¿Está en wait? | |

| (wait_remaining > 0) | |

|

| Sí | No |

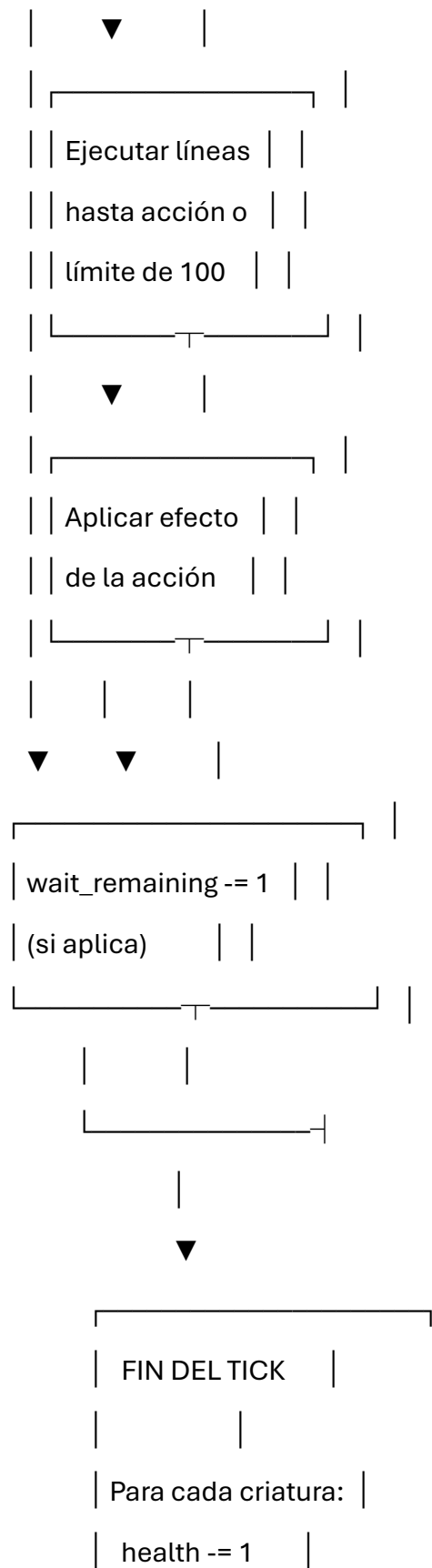


|

| Actualizar | |

| percepciones | |

|



```

| age += 1 |
| si muere → quitar |
|         |
| Para cada terreno: |
|   regenerar reserva |
|         |
| Para cada objeto: |
|   si reserve ≤ 0 |
|   → desaparece |
|_____|

```

8. Consejos Antes de Empezar a Codear

1. **Empieza por el lexer y parser** — sin esto nada funciona. Prueba que parsee correctamente los 4 archivos de ejemplo (hobbit, uruk, ent, dwarf).
2. **Luego el mundo sin criaturas** — carga terrenos, objetos, mapas. Verifica que puedes consultar casillas.
3. **Luego el intérprete sobre un mundo simple** — pon un hobbit solo en un mapa de pasto y verifica que ejecuta sus instrucciones correctamente.
4. **La GUI al final** — el PDF dice explícitamente: "Si no pueden correr una simulación completa desde la terminal, sin abrir visual, está mal separado." Primero que funcione todo sin GUI.
5. **Registros/catálogos, no cadenas de ifs** — para acciones, percepciones, funciones. Esto es lo que te va a dar la nota alta.
6. **Tests constantemente** — cada vez que implementes algo, pruébalo con los ejemplos del PDF.